

MediSynth: A Hospital Management System with a Privacy-Preserving Synthetic Patient Data Engine

Project Proposal for UCS503P

Submitted to: Nisha Thakur

Thapar Institute of Engineering and Technology

1. Higher-order goal

This project aims to improve healthcare data governance by removing a structural barrier that slows down software and AI development for hospitals: the unavailability of realistic, legally usable patient data. While the underlying problem is broader than any single hospital, the immediate engineering objective is to translate that governance need into a measurable, deployable software solution - a working hospital management system paired with a synthetic data engine that can safely stand in for real patient records.

2. Time-to-value

- We will deliver meaningful increments quickly by prototyping the core hospital workflow (patients, doctors, appointments) and the synthetic data pipeline in parallel, validating each independently within short iteration windows.
- We will prioritize fast feedback over perfection by using weekly iterations across our four build lanes (backend, frontend, mobile, data engine), with CI-backed automation introduced early to keep integration friction low.
- Success is defined by what can be delivered and measured in the first iteration - a working CRUD core fed by a first-pass synthetic dataset - then improved based on feedback and statistical validation.

3. Problem Statement

Hospital software teams, student projects, and healthcare researchers routinely need realistic patient data to build, test, and demonstrate systems - but real patient records are protected health information and cannot legally or ethically be shared for development, training, or classroom use. Common pain points include:

- **Legal restriction:** real records fall under HIPAA / India's DPDP Act; hospitals cannot hand them to developers, students, or third-party testers without extensive data-use agreements.
- **Re-identification risk:** simple anonymization (removing names/IDs) is frequently reversible by cross-referencing age, locality, and diagnosis, so "anonymized" data is not a safe substitute.

- **Unrealistic test data:** teams that avoid real data often fall back on small, hand-typed fake records that don't reflect real clinical patterns (age–diagnosis correlation, prescription frequency, appointment load), producing systems that are never properly stress-tested.
- **Fragmented development:** without a shared, realistic dataset, each lane of a hospital system (backend, frontend, mobile, analytics) tends to invent its own inconsistent mock data, causing integration rework.

Why this matters now (contextual relevance): as hospitals increasingly digitize and adopt AI-assisted tools, the demand for safe, realistic data to build and validate those tools is rising faster than the legal pathways to access real records - making a synthetic-data-first approach directly relevant.

4. Proposed Solution

4.1 Overview

Build a web- and mobile-accessible "Hospital-Management-plus-Synthetic-Data" system with the following components:

- **Hospital management core:** patient records, doctor profiles, appointment scheduling, admissions, and prescriptions, with role-based access for admins, doctors, and staff.
- **Synthetic data engine:** a Python-based pipeline that learns statistical patterns from reference healthcare data and generates new, artificial patient records that preserve those patterns without representing any real individual.
- **Audit and access layer:** JWT-based authentication, role-based authorization, and audit logging on all sensitive actions.
- **Multi-tenant isolation:** schema design that lets multiple hospitals use one deployment without any cross-visibility of data.
- **Cross-platform access:** a React web frontend for administrative/clinical use and a Flutter mobile app for on-the-go access.

4.2 Core workflow

1. Synthetix generates a synthetic patient dataset from reference statistical patterns (age–diagnosis distributions, prescription correlations, appointment rhythms).
2. The hospital management system is seeded with this synthetic data, giving every lane of the project (backend, frontend, mobile) a consistent, realistic dataset to build against.
3. Staff and doctors interact with the system exactly as they would with real data - creating appointments, updating records, issuing prescriptions - under full role-based access control.

4. Every sensitive action is captured in an audit log, and the synthetic engine can regenerate fresh datasets on demand for further testing without any privacy exposure.

4.3 Operational constraints for an educational setting

- Keep the management system usable on standard college-lab and mid-range mobile hardware.
- Start the synthetic engine with explainable, statistical methods (distribution sampling, correlation preservation) rather than research-grade generative models; treat deep generative models (GANs/VAEs) as a stretch goal only.
- Design for deployability using common managed database and hosting services.

5. Solution Approach

This is an engineering course project, so we focus on system correctness, data-pipeline design, and integration across four coordinated lanes, rather than novel generative-modeling research.

5.1 Synthetic data generation

- Profile reference/public healthcare statistics (age bands, disease prevalence, prescription–diagnosis correlation) to define generation distributions.
- Generate synthetic patient, appointment, and prescription records that reproduce these statistical patterns using a CSV-to-synthetic-CSV pipeline (starting with rule-based sampling, extendable to library-based statistical modeling such as SDV).
- Validate synthetic output against reference distributions (e.g., compare summary statistics) rather than claiming state-of-the-art generative fidelity.

5.2 System architecture

- **Frontend:** React web application for admin, doctor, and staff dashboards.
- **Mobile:** Flutter app for lightweight patient/staff access on the go.
- **Backend API:** Node.js service handling authentication, authorization, CRUD operations, and audit logging.
- **Database:** PostgreSQL with a 13-table schema and multi-tenant isolation.
- **Data engine:** Python-based Synthetix pipeline, decoupled from the backend so it can be re-run independently to refresh datasets.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to automatically build and test each of the four lanes on every change, produce repeatable deployment artifacts to a staging environment, and enable frequent, safe releases of incremental features across the 10-week plan.

6. Evaluation Criterion

We define success using measurable metrics tied to both the management system and the synthetic data engine.

6.1 Primary evaluation metrics

- **System correctness:** percentage of core workflows (patient registration, appointment booking, prescription issuance) completing without error against the seeded synthetic dataset.
- **Statistical fidelity of synthetic data:** similarity between key summary statistics (e.g., age–diagnosis distribution, appointment volume patterns) of the generated dataset and the reference distributions it was modeled on.

6.2 Secondary evaluation metrics

- Privacy guarantee: zero real-patient records present anywhere in the generated dataset or the seeded database (verifiable by construction, since generation never starts from real records).
- Generation performance: time to produce a full synthetic dataset of a target size (e.g., 10,000 patient records).
- Role-based access correctness: percentage of access-control test cases (admin/doctor/staff boundaries) passing.
- Reliability: staging environment availability during development and demo windows (e.g., $\geq 99\%$ uptime).

6.3 Validation plan

- Generate and inspect multiple synthetic dataset batches, comparing distributions against reference statistics before seeding the application.
- Run the three-person team through end-to-end role scenarios (admin, doctor, staff) against the seeded system to confirm workflow correctness.

7. Scalability

The proposition should scale in both theoretical and practical senses.

1. Scalable (theoretically): The system should support growth in patient volume, hospitals (tenants), and dataset size by:

- decoupling the Synthetix engine from the backend so datasets can be regenerated or scaled independently,
- using multi-tenant schema isolation so additional hospitals can be onboarded without redesign,

- enabling pagination and indexing for common queries on large synthetic datasets.

2. Operationally deployable (practically): The system should run on common web hosting:

- managed PostgreSQL database,
- containerized backend and data-engine deployment,
- CI/CD pipeline for staging/production across all four lanes.

8. Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a full-stack, multi-lane project:

- Web framework for REST APIs (Node.js/Express) and a reactive frontend (React).
- Cross-platform mobile framework (Flutter).
- Managed relational database (PostgreSQL).
- Token-based authentication (JWT) with role-based authorization.
- Python data/ML ecosystem (pandas, statistical/generative libraries such as SDV) for the Synthetix engine.
- Test framework for unit/integration tests, plus a CI runner and staging deployment target.

We will use these mature components to focus on data-pipeline design, workflow correctness, and measurement rather than inventing new infrastructure.

9. Project scope and deliverables

9.1 Initial deliverable ... first iteration

- Core database schema (13 tables) with multi-tenant isolation
- JWT authentication and role-based authorization on the backend
- Synthetix v1: rule-based/statistical CSV-to-synthetic-CSV generator for patient records
- Basic patient/doctor/appointment CRUD on backend and frontend, seeded with synthetic data
- Audit logging on sensitive actions
- CI: build + backend unit tests + linting across lanes

9.2 Subsequent deliverables

- Extended synthetic generation: prescriptions, admissions, billing records with cross-table statistical consistency
- Flutter mobile app covering patient/staff-facing workflows

- Admin/ops dashboard for hospital-level metrics and dataset regeneration
- Statistical fidelity reporting for generated datasets (distribution comparisons)
- Stretch goal: generative-model-based synthesis (e.g., GAN/VAE) for higher-fidelity edge cases

10. Risks and mitigations

- **Statistical fidelity risk:** synthetic data may fail to capture rare/edge-case patterns; mitigate by explicitly modeling known edge cases and validating against reference distributions before each release.
- **Integration risk across four lanes:** backend, frontend, mobile, and data-engine work happening in parallel can drift out of sync; mitigate with a shared schema contract, weekly integration checkpoints, and CI across all lanes.
- **Data privacy / consent (for any reference data used):** use only public/aggregate reference statistics to shape generation, never real patient-level records; keep the synthetic engine fully decoupled from any real data source.
- **Team bandwidth (3-person team, 10-week plan):** sequence the 21-phase plan so each lane has a clear owner and dependencies are made explicit early to avoid late-stage bottlenecks.

11. Summary

This project proposes MediSynth, a deployable hospital management system paired with Synthetix, a synthetic patient data engine that removes the legal and practical barrier of needing real patient records for development and testing. It meets the course criteria by emphasizing time-to-value through parallel, iterative delivery across four build lanes, using CI/CD to support frequent safe releases, and defining measurable evaluation criteria — particularly statistical fidelity of generated data and system workflow correctness. It remains focused on engineering system design, data-pipeline correctness, and scalability readiness, with generative-modeling novelty treated as a stretch goal rather than the core deliverable.

Name - Aryan Sharma , **Roll No:** 1024030789

Name - Brahmdeep Singh Khanuja , **Roll No:** 1024030792

Name - Keerat Singh Brar , **Roll No:** 1024030793